

Code:

```
#include <stdio.h> #include <stdlib.h> #include <string.h>

#define MAXN 100005

typedef struct { int to, next; } Edge;

Edge e1[MAXN * 2], e2[MAXN * 2]; int head1[MAXN], head2[MAXN], cnt1, cnt2; int n;

void add_edge1(int u, int v) { e1[++cnt1] = (Edge){v, head1[u]}; head1[u] = cnt1; }

void add_edge2(int u, int v) { e2[++cnt2] = (Edge){v, head2[u]}; head2[u] = cnt2; }

int cmp_int(const void a, const void b) { return ((int)a - (int)b); }

// Tree hashing using deterministic subtree vector sorting
int hash_tree1(int u, int p) { int child_hashes[1000]; int deg = 0; for (int i = head1[u]; i; i = e1[i].next) { int v = e1[i].to; if (v != p) { child_hashes[deg++] = hash_tree1(v, u); } } qsort(child_hashes, deg, sizeof(int), cmp_int); unsigned long long h = 2166136261u; for (int i = 0; i < deg; i++) { h = (h ^ child_hashes[i]) * 16777619u; } return (int)(h ^ (deg * 313)); }

int hash_tree2(int u, int p) { int child_hashes[1000]; int deg = 0; for (int i = head2[u]; i; i = e2[i].next) { int v = e2[i].to; if (v != p) { child_hashes[deg++] = hash_tree2(v, u); } } qsort(child_hashes, deg, sizeof(int), cmp_int); unsigned long long h = 2166136261u; for (int i = 0; i < deg; i++) { h = (h ^ child_hashes[i]) * 16777619u; } return (int)(h ^ (deg * 313)); }

int main() { int t; if (scanf("%d", &t) != 1) return 0;

while (t--) {
    if (scanf("%d", &n) != 1) break;
    cnt1 = cnt2 = 0;
    for (int i = 1; i <= n; i++) {
        head1[i] = head2[i] = 0;
    }

    for (int i = 0; i < n - 1; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        add_edge1(u, v);
        add_edge1(v, u);
    }
    for (int i = 0; i < n - 1; i++) {
```

```
        int u, v;
        scanf("%d %d", &u, &v);
        add_edge2(u, v);
        add_edge2(v, u);
    }

    int h1 = hash_tree1(1, 0);
    int h2 = hash_tree2(1, 0);

    if (h1 == h2) printf("YES\n");
    else printf("NO\n");
}
return 0;

}
```

Input:

```
2
3
1 2
2 3
1 2
1 3
3
1 2
2 3
1 3
3 2
```

Output:

```
NO

YES
```